

TOPICS INCLUDED

- Recent trends in the Industry, Introduction of Cloud Database, Introduction of Distributed Database, Introduction to NOSQL Database and Object-Oriented Databases.
- Learn about Emerging Database Technologies. Explore different NoSQL Types. Learn how Object-Oriented Programming Concepts like Objects and Inheritance are applied to Database Management Systems.

6.1 RECENT TRENDS IN THE INDUSTRY

- The Database Management Systems (DBMS) have come a lot way in their development since their introduction, which reflects the state of the technology as well as the needs of the industry.
- DBMS technology has made a long way from the early hierarchical databases to the current sophisticated, cloud-based systems, and it continues to be a driver of the demands for scalability, performance, and flexibility.
- With this article, we will study the historical growth of DBMS and we will also discuss the latest trends that are influencing the future of database technology.

6.1.1 Historical Overview of DBMS Evolution

1. Hierarchical and Network Databases (1960s - 1970s):

(a) **Hierarchical Databases:** The first DBMS models such as IBM's Information Management System (IMS), used to organize data in a tree-like structure. These systems proved to be effective for specific tasks, but they found it difficult to tackle complicated data relationships.

(b) **Databases:** The models like the Integrated Data Store (IDS) introduced more flexible data relationships through a network structure, allowing multiple parent-child relationships. They, nevertheless, were the ones who were still restricted by the rigid structures and complex navigational queries.

2. Relational Databases (1970s - 1980s):

(a) **SQL and Relational Model:** The relational model of Edgar F. Codd that was introduced in 1970 has revolutionized database management. DBs of Relational used tables to show data and SQL (Structured Query Language) to query and change

it. This method not only made data management easier but also increased data integrity and allowed for more complex queries.

(b) **Commercial Adoption:** Companies such as Oracle, IBM (with DB2), and Microsoft (with SQL Server) were among the first ones to commercialize relational databases, which became the standard for data management in business applications.

3. Object-Oriented and Post-Relational Databases (1980s - 1990s):

(a) **Object-Oriented Databases:** Technology was so advanced by now that they managed to benefits from object-oriented principles and implemented database that is composed of objects rather than rows and columns. They were designed to handle complex data and relationships but faced challenges with adoption and integration with existing systems.

(b) **Post-Relational Databases:** Congestion of data requests became a problem when the requirements of the data grew, hence the development of the hybrid models combining object-oriented and relational models to solve them with the traditional relational systems.

4. NoSQL and NewSQL Databases (2000s - 2010s):

(a) **NoSQL Databases:** The advent of big data and horizontal scalability requirements that drove the growth of NoSQL databases in the form of document stores, key-value stores, column-family stores, and graph databases, for example, MongoDB, Redis, Cassandra, and Neo4j, correspondingly. Such systems were much more flexible in modeling data and showed better performance in big-scale applications.

(b) **NewSQL databases:** were designed to fill the gap between traditional relational databases and NoSQL systems, marrying relational features with NoSQL systems' scalability and performance.

6.1.2 Recent Trends in DBMS

1. Cloud-Native Databases:

- (i) **Managed Services:** The popularity of cloud computing has, therefore, resulted in increasing recourse to managed DBMS services from providers like AWS, Microsoft Azure, and GCP. Such services will help minimize the burden of managing databases with activities like scaling, backups, and patching, so the business can remain focused on its core activities.
- (ii) **Serverless Databases:** It is the serverless architecture, as in the cases of AWS Aurora Serverless and Google Cloud Spanner, that provides support for auto-scaling and on-demand resource allocation without manual provisioning and management.

2. Hybrid and Multi-Cloud Strategies:

- **Data Distribution:** Most hybrid and multi-cloud strategies include improved data distribution, better redundancy, and avoiding vendor lock-in. It is such a move that will integrate seamlessly both on-premises databases and cloud-based systems.

3. Artificial Intelligence and Machine Learning Integration:

- **AI-Powered Insights:** State-of-the-art Database Management Systems are integrating Artificial Intelligence and machine learning capabilities for advanced analytics, predictive modeling, and automated data management. AI adds features, such as automated indexing, anomaly detection, and query optimization to enhance performance and operational efficiencies.

4. Real-Time Data Processing:

- **Streaming data:** Due to the need for real-time processing of data, technologies like Apache Kafka and Apache Flink have been realized in bringing the real-time analytics coupled with event-driven architectures. These systems underpin applications that require very low tolerances for data insights to be generated, such as applications in fraud detection and real-time recommendation engines.

5. Enhanced Security and Privacy:

- **Data Protection:** As breaches are getting more frequent and other privacy concerns grow, most modern DBMS platforms turn to optimize their security features, including encryption, access controls, and compliance adherence, such as GDPR and CCPA. Features related to data masking and at-rest and in-transit encryption are getting standardized.

6. Graph Databases and Semantic Web Technologies:

- **Graph Databases:** As the world's data becomes ever more interconnected, interest in graph databases grows, since they are exceptionally good at handling and querying complicated relationships. This makes them very useful in applications such as social networks, recommendation engines, and knowledge graphs for modelling and querying intricate relationships.

7. Blockchain and Distributed Ledger Technology:

- **Immutable Data Storage:** It is the area of study regarding potentials for secure and immutable data storage in blockchain technology. Distributed ledger technologies have opened fresh opportunities for applications requiring transparent and non-tamperable data management in domains such as financial transactions and supply chain tracking.

6.2 INTRODUCTION OF CLOUD DATABASE

- A cloud database is a database that is deployed, hosted, and accessed on a cloud computing platform (public, private, or hybrid) instead of on local on-premise servers.
- It is accessed over the internet and is typically offered as a managed service by providers like AWS, Azure, and Google Cloud.

Cloud Database:

- A cloud database is a collection of organized data that runs on cloud infrastructure and is accessed as a service (self-managed on VMs or fully managed DBaaS).
- It can be relational (SQL) or non-relational (NoSQL) and usually provides built-in scalability, high availability, back-up, and security features handled by the provider.
- **Examples:** Amazon RDS, Azure SQL Database, Google Cloud SQL (relational); MongoDB Atlas, Amazon DynamoDB (NoSQL).

6.2.1 Features of Cloud Databases

- Cloud databases provide managed, internet-accessible storage with built-in scalability, availability, and security, making them ideal for modern web, mobile, and analytics workloads.
- Below are key **features** with detailed descriptions and concrete examples.
- (i) **On-demand scalability:**
 - A cloud database can scale vertically (more CPU/RAM) and horizontally (more replicas/partitions) without buying new hardware. Scaling can be automatic (based on CPU, connections, or storage) or manual via console/API

Example: An e-commerce site on **Amazon RDS for MySQL** increases instance size and storage automatically during a Diwali sale, then scales down after the sale to reduce cost.

(ii) High availability and fault tolerance:

- Data is replicated across availability zones or regions, and automatic failover promotes a standby if the primary instance fails. This reduces downtime from hardware failures, outages, or maintenance.

- Example:** **Google Cloud SQL** uses a primary and standby instance in different zones; if the primary crashes, the standby becomes primary with minimal interruption to a production ERP system.

(iii) Automated backups and disaster recovery:

- Cloud DBs provide scheduled backups, on-demand snapshots, and point-in-time restore, often stored in durable object storage. This protects against accidental deletes, corruption, or ransomware.

- Example:** In **Azure SQL Database**, daily automatic backups allow a college MIS to restore the database to a specific time just before a mistaken bulk delete of student records.

(iv) Pay-as-you-go pricing:

- Charges are based on actual usage (compute hours, storage GB, I/O, back-up space) instead of large upfront capital expenses. Some services add serverless or "pause when idle" modes to save further.

- Example:** A startup uses **Amazon Aurora Serverless** for a prototype; the database automatically pauses at night and resumes on first request, so they pay only for active usage.

(v) Managed maintenance and updates:

- The provider handles OS/DB engine patching, minor version upgrades, monitoring, and sometimes index or configuration tuning. This reduces DBA workload and security risk from unpatched systems.

- Example:** An EdTech platform on **Azure SQL Database** receives automatic security and engine updates during maintenance windows without manual intervention, letting the small tech team focus on features instead of patching.

(vi) Security, compliance, and access control:

- Cloud databases offer encryption at rest and in transit, IAM roles, network isolation (VPC/VNet, private endpoints), auditing, and compliance

certifications (for example, GDPR, HIPAA, PCI-DSS). Fine-grained access control and logging help meet regulatory requirements.

- Example:** A healthcare SaaS uses **MongoDB Atlas** with encrypted storage, IP whitelisting, and role-based access to securely store patient records in a HIPAA-aligned environment.

(vii) Global accessibility and distributed access:

- Applications and users anywhere can connect over the internet or private links, and some cloud DBs support multi-region replication for low-latency global access.

- Example:** **Azure Cosmos DB** replicates data to Europe and Asia for a global social networking app, so reads and writes are served from the nearest region to reduce latency for users in Mumbai, London, and Singapore.

(viii) Flexible data models and service choices:

- Cloud platforms provide relational, NoSQL, and analytical databases, often as managed services (DBaaS) with different consistency, schema, and performance characteristics. This lets architects choose the right tool per workload.

- Example:** A logistics company uses **PostgreSQL on Amazon RDS** for transactional data (orders, invoices) and **Amazon DynamoDB** for high-throughput tracking events, integrating both in a single cloud solution.

6.2.2 Benefits of cloud database

- Cloud databases offer significant benefits over traditional on-premises databases by reducing costs, improving performance, and enhancing reliability through cloud-native capabilities.

(i) Cost Efficiency:

- Cloud databases eliminate upfront hardware purchases and reduce ongoing maintenance expenses via. pay-as-you-go pricing based on actual usage (storage, compute, I/O)
- A small E-Commerce start-up migrates from self-hosted MySQL servers (≠ 10L initial cost + ≠ 2L/year maintenance) to Amazon RDS, paying only ≠ 5K/month initially and scaling as sales grow without capital expenditure.

(ii) Scalability and Elasticity:

- Resources scale automatically or on-demand to handle traffic spikes without downtime or over-provisioning.
- During Black Friday, a retail app on Google Cloud SQL auto-scales from 2 vCPUs to 16 vCPUs, serving 10x queries while a traditional DB would crash or require weeks of hardware upgrades.

(iii) High Availability and Reliability:

- Built-in replication across zones/regions ensures 99.99%+ uptime with automatic failover and load balancing.
- A banking app on Azure SQL Database survives a data center outage as replicas in multiple regions take over seamlessly, maintaining zero transaction loss unlike single-server on-prem setups.

(iv) Reduced Operational Overhead:

- Providers handle patching, backups, monitoring, and upgrades, freeing IT teams for business logic.
- An EdTech firm running Oracle on AWS shifts from 2 full-time DBAs (managing backups/patches) to zero, focusing on app development while the provider ensures 24/7 security updates.

(v) Enhanced Security and Compliance:

- Encryption (at rest/transit), IAM, VPC isolation, and certifications (HIPAA, PCI-DSS, GDPR) exceed many on-prem set-ups.
- A healthcare provider stores patient data in MongoDB Atlas with end-to-end encryption and audit logs, easily passing HIPAA audits that would require expensive custom security for on-premises MySQL.

(vi) Disaster Recovery and Data Protection:

- Automated backups, point-in-time recovery, and cross-region replication minimize RPO/RTO to minutes.
- After ransomware hits a manufacturing firm's on-prem SQL Server (requiring 3-day restore from tape), they switch to Snowflake with daily snapshots, recovering in 15 minutes post-incident.

(vii) Global Accessibility and Collaboration:

- Data accessible from anywhere with low-latency multi-region replication for distributed teams.
- A global SaaS CRM uses Azure Cosmos DB replicated across India, US, and Europe, enabling real-time sales updates for Mumbai and New York teams without VPN complexities of on-premDBs.

(viii) Faster Time-to-Market and Agility:

- Provision databases in minutes for prototyping, testing, and production without procurement delays
- A hackathon team launches a full-stack app with PostgreSQL on Heroku Postgres in 10 minutes vs. weeks waiting for server approval/purchase in traditional IT set-ups.

6.2.3 Working of Cloud Databases

- In businesses, databases are used to collect, organize and deliver data to executives and workers for operational and analytics applications. In general, cloud databases provide the same data processing, management and access capabilities as on-premises ones. Existing on-premises databases usually can be migrated to the cloud, along with the applications they support.
- Instead of traditional software licenses, pricing is based on the use of system resources, which can be provisioned on demand as needed to meet processing workloads. Alternatively, users can reserve database instances -- typically for at least a year -- to get discounted pricing on regular workloads with consistent capacity requirements.
- Organizations that are implementing databases in the public cloud choose between the following two deployment models:

(A) Self-managed database:

- This is an infrastructure as a service (IaaS) environment, in which the database runs in a virtual machine on a system operated by a cloud provider.
- The provider manages and supports the cloud infrastructure, including servers, operating systems and storage devices. But the user organization is responsible for database deployment, administration and maintenance. As a result, it's akin to an on-premises deployment for the DBA, who retains full management control of the database.

(B) Managed Database Service:

- Database as a service (DBaaS) environments are fully managed by the vendor, which could be a cloud platform provider or another database vendor that runs its cloud DBMS on a platform provider's infrastructure. Under the DBaaS model, both the system infrastructure and the database platform are managed for the customer. The DBaaS vendor handles provisioning, backups, scaling, patching, upgrades and other basic database administration functions, while the DBA monitors the database and coordinates with the vendor on some administrative tasks. Similar data warehouse as a service (DWaaS) offerings are also available for deployments of cloud data warehouses.
- In addition, some cloud providers -- Amazon Web Services (AWS) and Oracle, for example -- offer versions of their DBaaS technologies for

(iii) High Availability and Reliability:

- Built-in replication across zones/regions ensures 99.99%+ uptime with automatic failover and load balancing.
- A banking app on Azure SQL Database survives a data center outage as replicas in multiple regions take over seamlessly, maintaining zero transaction loss unlike single-server on-prem setups.

(iv) Reduced Operational Overhead:

- Providers handle patching, backups, monitoring, and upgrades, freeing IT teams for business logic.
- An EdTech firm running Oracle on AWS shifts from 2 full-time DBAs (managing backups/patches) to zero, focusing on app development while the provider ensures 24/7 security updates.

(v) Enhanced Security and Compliance:

- Encryption (at rest/transit), IAM, VPC isolation, and certifications (HIPAA, PCI-DSS, GDPR) exceed many on-prem set-ups.
- A healthcare provider stores patient data in MongoDB Atlas with end-to-end encryption and audit logs, easily passing HIPAA audits that would require expensive custom security for on-premises MySQL.

(vi) Disaster Recovery and Data Protection:

- Automated backups, point-in-time recovery, and cross-region replication minimize RPO/RTO to minutes.
- After ransomware hits a manufacturing firm's on-prem SQL Server (requiring 3-day restore from tape), they switch to Snowflake with daily snapshots, recovering in 15 minutes post-incident.

(vii) Global Accessibility and Collaboration:

- Data accessible from anywhere with low-latency multi-region replication for distributed teams.
- A global SaaS CRM uses Azure Cosmos DB replicated across India, US, and Europe, enabling real-time sales updates for Mumbai and New York teams without VPN complexities of on-premDBs.

(viii) Faster Time-to-Market and Agility:

- Provision databases in minutes for prototyping, testing, and production without procurement delays
- A hackathon team launches a full-stack app with PostgreSQL on Heroku Postgres in 10 minutes vs. weeks waiting for server approval/purchase in traditional IT set-ups.

6.2.3 Working of Cloud Databases

- In businesses, databases are used to collect, organize and deliver data to executives and workers for operational and analytics applications. In general, cloud databases provide the same data processing, management and access capabilities as on-premises ones. Existing on-premises databases usually can be migrated to the cloud, along with the applications they support.
- Instead of traditional software licenses, pricing is based on the use of system resources, which can be provisioned on demand as needed to meet processing workloads. Alternatively, users can reserve database instances -- typically for at least a year -- to get discounted pricing on regular workloads with consistent capacity requirements.
- Organizations that are implementing databases in the public cloud choose between the following two deployment models:

(A) Self-managed database:

- This is an infrastructure as a service (IaaS) environment, in which the database runs in a virtual machine on a system operated by a cloud provider.
- The provider manages and supports the cloud infrastructure, including servers, operating systems and storage devices. But the user organization is responsible for database deployment, administration and maintenance. As a result, it's akin to an on-premises deployment for the DBA, who retains full management control of the database.

(B) Managed Database Service:

- Database as a service (DBaaS) environments are fully managed by the vendor, which could be a cloud platform provider or another database vendor that runs its cloud DBMS on a platform provider's infrastructure. Under the DBaaS model, both the system infrastructure and the database platform are managed for the customer. The DBaaS vendor handles provisioning, backups, scaling, patching, upgrades and other basic database administration functions, while the DBA monitors the database and coordinates with the vendor on some administrative tasks. Similar data warehouse as a service (DWaaS) offerings are also available for deployments of cloud data warehouses.
- In addition, some cloud providers -- Amazon Web Services (AWS) and Oracle, for example -- offer versions of their DBaaS technologies for

installation in on-premises data centers as part of a private cloud or a hybrid cloud infrastructure that combines public and private clouds. As with a regular DBaaS environment, the provider deploys the databases on its own systems and manages them for customers. But it delivers the systems to a customer's data center to run there and then manages the databases remotely.

- Many vendors now also offer serverless databases in the cloud. Like DBaaS, they are managed services, and the two terms are sometimes used interchangeably. But there are some differences. For example, serverless systems automatically provide the processing resources required by database applications and scale-up or down as workloads fluctuate, while DBaaS commonly includes a specific amount of resources with scaling options. The term serverless is really a misnomer -- the databases do run on a cloud provider's servers. But they're effectively serverless from a customer's standpoint.

6.2.4 Types of Cloud Databases

A wide variety of cloud databases are available, matching the different types of database technologies that can be deployed on premises. At this point, every notable database vendor offers its software in the cloud. That includes cloud-native databases developed specifically for use in cloud environments and existing on-premises databases that now support the cloud.

(A) Relational Databases:

- Relational software that's accessed and managed with the SQL programming language has dominated the database market since the 1990s and remains the most widely used DBMS technology.
- It's particularly well suited for transaction processing and other applications involving structured data, thanks to the relational model's support for data integrity and consistency. Relational databases organize data in tables with rows and columns and use a fixed schema to help enforce consistency rules.

(B) NoSQL databases:

- NoSQL systems forego the rigid schemas of relational databases, making them a better option for semistructured and unstructured data, such as text, log files, sensor data and videos.
- Despite the term, many do provide some SQL capabilities and can also be used to store large amounts of structured data, particularly for applications that don't require complete consistency.

- Because of that, NoSQL more commonly means "not only SQL" now. There are four major NoSQL product categories: document databases, key-value databases, wide-column stores and graph databases.

(C) Multimodel databases:

- These databases support more than one data model, enabling them to run a wider set of applications. Many relational and NoSQL databases now qualify as multimodel through add-ons
- For example**, the addition of a graph module to a relational DBMS or a NoSQL document database.

(D) Distributed SQL databases:

- Initially labelled as NewSQL databases and still referred to by that name in some cases, these technologies distribute relational databases across multiple computing nodes to create transactional systems that can provide NoSQL-like levels of scalability.

(E) Cloud data warehouses:

- First developed to provide data warehousing capabilities for business intelligence and reporting applications, cloud data warehouse and DWaaS technologies typically now also support development of data lakes that contain large amounts of raw data, as well as machine learning and other advanced analytics functions.
- Specialized databases** are also available for particular applications. Most notably, they include time series databases that hold time-stamped data stored in sequential order; vector databases designed to support large-scale similarity searches on sets of unstructured data; more conventional database search engines; and ledger databases that create an immutable record of transactions using blockchain and other cryptographic techniques.

6.3 INTRODUCTION OF DISTRIBUTED DATABASE

- A distributed database is a database system that spans multiple computers or nodes that are connected by a network. Each node in a distributed database can store a portion of the data, and the entire database is made up of the sum of the data stored on each node.
- In a distributed database, data is stored and processed in a distributed manner, and the system ensures that the data remains consistent and available to users despite network failures or other system errors. The

installation in on-premises data centers as part of a private cloud or a hybrid cloud infrastructure that combines public and private clouds. As with a regular DBaaS environment, the provider deploys the databases on its own systems and manages them for customers. But it delivers the systems to a customer's data center to run there and then manages the databases remotely.

Many vendors now also offer serverless databases in the cloud. Like DBaaS, they are managed services, and the two terms are sometimes used interchangeably. But there are some differences. For example, serverless systems automatically provide the processing resources required by database applications and scale-up or down as workloads fluctuate, while DBaaS commonly includes a specific amount of resources with scaling options. The term serverless is really a misnomer -- the databases do run on a cloud provider's servers. But they're effectively serverless from a customer's standpoint.

6.2.4 Types of Cloud Databases

A wide variety of cloud databases are available, matching the different types of database technologies that can be deployed on premises. At this point, every notable database vendor offers its software in the cloud. That includes cloud-native databases developed specifically for use in cloud environments and existing on-premises databases that now support the cloud.

(A) Relational Databases:

- Relational software that's accessed and managed with the SQL programming language has dominated the database market since the 1990s and remains the most widely used DBMS technology.
- It's particularly well suited for transaction processing and other applications involving structured data, thanks to the relational model's support for data integrity and consistency. Relational databases organize data in tables with rows and columns and use a fixed schema to help enforce consistency rules.

(B) NoSQL databases:

- NoSQL systems forego the rigid schemas of relational databases, making them a better option for semistructured and unstructured data, such as text, log files, sensor data and videos.
- Despite the term, many do provide some SQL capabilities and can also be used to store large amounts of structured data, particularly for applications that don't require complete consistency.

- Because of that, NoSQL more commonly means "not only SQL" now. There are four major NoSQL product categories: document databases, key-value databases, wide-column stores and graph databases.

(C) Multimodel databases:

- These databases support more than one data model, enabling them to run a wider set of applications. Many relational and NoSQL databases now qualify as multimodel through add-ons.
- For example**, the addition of a graph module to a relational DBMS or a NoSQL document database.

(D) Distributed SQL databases:

- Initially labelled as NewSQL databases and still referred to by that name in some cases, these technologies distribute relational databases across multiple computing nodes to create transactional systems that can provide NoSQL-like levels of scalability.

(E) Cloud data warehouses:

- First developed to provide data warehousing capabilities for business intelligence and reporting applications, cloud data warehouse and DWaaS technologies typically now also support development of data lakes that contain large amounts of raw data, as well as machine learning and other advanced analytics functions.
- Specialized databases** are also available for particular applications. Most notably, they include time series databases that hold time-stamped data stored in sequential order; vector databases designed to support large-scale similarity searches on sets of unstructured data; more conventional database search engines; and ledger databases that create an immutable record of transactions using blockchain and other cryptographic techniques.

6.3 INTRODUCTION OF DISTRIBUTED DATABASE

- A distributed database is a database system that spans multiple computers or nodes that are connected by a network. Each node in a distributed database can store a portion of the data, and the entire database is made up of the sum of the data stored on each node.
- In a distributed database, data is stored and processed in a distributed manner, and the system ensures that the data remains consistent and available to users despite network failures or other system errors. The

primary goal of a distributed database is to provide high availability, scalability, and performance to applications that require access to large amounts of data.

6.3.1 Need of Distributed Database in DBMS

- A database is an structured collection of information. The data can be easily accessed, managed, modified, updated, controlled, and organized in a database.
- Databases can be broadly classified into two types, namely **Distributed** and **Centralized** databases. The question here is why do we even need a Distributed Database In Dbms?. Let us assume for a moment that we have only centralized databases.
 - We will be inserting all the data into one single database. Making it too large so that it will take a lot of time to query a single piece of record.
 - Once a fault occurs, we no longer be able to serve user requests as we have only one database.
 - No scaling is possible even if we wanted to and availability is also less which in turn affects the throughput.
- Distributed databases **resolve various issues**, such as *availability, fault tolerance, throughput, latency, scalability*, and many other problems that can arise from using a single machine and a single database. **That is why we need distributed databases.** Let us discuss them in detail.

6.3.2 Distributed Databases

- A **distributed database** is a database that is not limited to one computer system. It is like a database that consists of two or more files located in different computers or sites either on the same network or on an entirely different network.
- These sites do not share any physical component. Distributed databases are needed when a particular data in the database needs to be accessed by various users globally. It needs to be handled in such a way that for a user it always looks like one single database.
- By contrast, a **Centralized database** consists of a single database file located at one site using a single network.
- Below is a reference diagram for distributed databases.

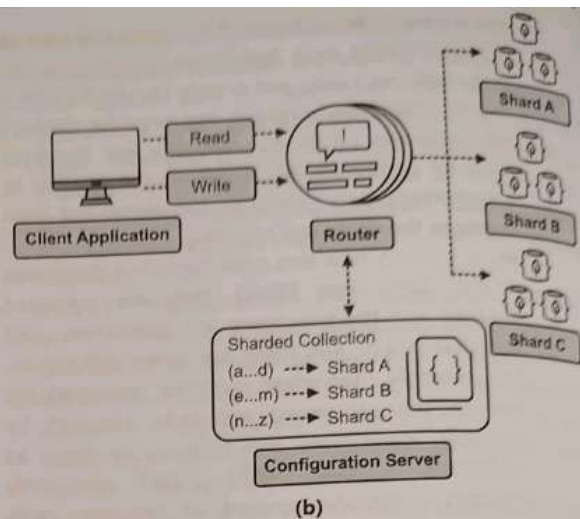
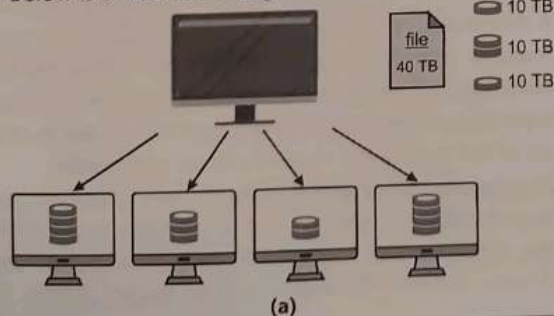


Fig. 6.1: Distributed Database

- Though there are many distributed databases to choose from, some examples of distributed databases include *Apache Ignite, Apache Cassandra, Apache HBase, Amazon SimpleDB, Clusterpoint*, and *FoundationDB*.

6.3.3 Features of Distributed Databases

- In general, distributed databases include the following features:
 1. **Location independency:** Data is independently stored at multiple sites and managed by independent *Distributed database management systems (DDBMS)*.
 2. **Network linking:** All distributed databases in a collection are *linked by a network* and communicate with each other.
 3. **Distributed query processing:** Distributed query processing is the procedure of answering queries (which means mainly read operations on large data sets) in a distributed environment.
 4. Query processing involves the transformation of a **high-level query** (for example, *formulated in SQL*) into a **query execution plan** (consisting of *lower-level query operators in some variation of relational algebra*) as well as the execution of this plan.
 5. **Hardware independent:** The different sites where data is stored are *hardware-independent*. There is no physical contact between these Distributed Database InDbms which is accomplished often through virtualization.
 6. **Distributed transaction management:** Distributed Database InDbms provides a consistent distribution through *commit protocols, distributed*

recovery methods, and distributed concurrency control techniques in case of many transaction failures.

6.3.4 Types of Distributed Database in DBMS

There are two types of distributed databases:

- **Homogenous distributed database.**
- **Heterogeneous distributed database.**

(A) Homogenous Distributed Database:

A **Homogenous distributed database** is a network of **identical databases** stored on multiple sites. All databases stores data identically, the operating system, DDBMS and the data structures used – all are same at all sites, making them **easy to manage**.

Below is a diagram for the same,

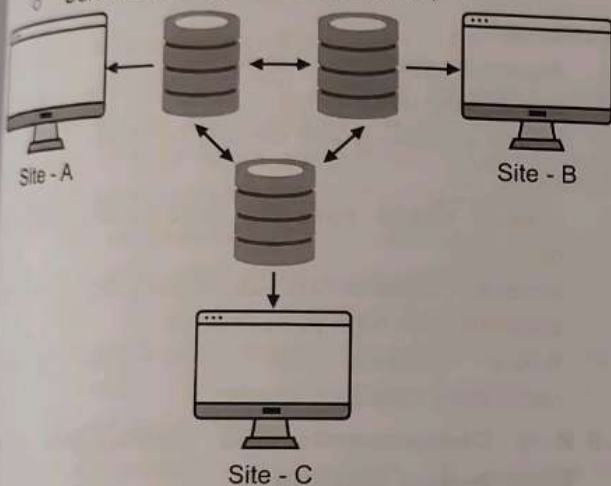


Fig. 6.2: Homogenous Distributed Database

In a homogeneous distributed database, the machines, nodes, servers, or sites store the same data, use the same data model, work with the same operating system, and share the same distributed database management system (DDBMS) or occasionally multiple types of DDBMS from the same vendor.

Within homogenous distributed databases, there are two subsets: autonomous and non-autonomous.

◦ **Autonomous distributed databases:** In an autonomous distributed database, nodes work on their own with their own complete set of data, only requiring an application to facilitate universal updates across all nodes or messaging between nodes.

◦ **Non-autonomous distributed databases:** In non-autonomous distributed databases, nodes rely on a centralized database management system (DBMS) to coordinate data distribution, communications, and all updates.

Features:

- Unified query language and interface.
- Low integration complexity.
- Efficient synchronization.

Example: A bank with branches in different cities uses Oracle DB at every location. All databases have the same structure and are synchronized regularly.

(B) Heterogeneous Distributed Database:

It is the opposite of a Homogenous distributed database. It uses *different schemas, operating systems, DDBMS, and different data models* causing it **difficult to manage**.

In the case of a Heterogeneous distributed database, a particular site can be **completely unaware** of other sites. This causes limited cooperation in processing user requests, this is why translations are required to establish communication between sites.

Below is a diagram for the same:

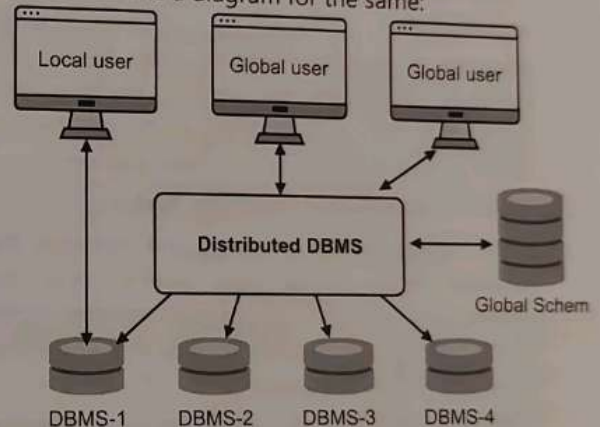


Fig. 6.3: Heterogeneous Distributed Database

◦ **Note:** Heterogeneous DDMS have local users while Homogenous DDMS does not have local users

◦ In a heterogeneous distributed database, different machines or sites may house different data sets, use different operating systems, contain different data schemas, and require software to facilitate communication between machines. Further, different sites may not even be aware of the existence of other sites.

◦ Within heterogeneous distributed databases, there are two subsets: federated and unfederated.

▪ **Federated Distributed Databases:** In a federated distributed database, multiple nodes — which are able to function completely on their own and may contain different data — can work together and function as one entity. This means that when a query occurs, the

system determines which node is best equipped to respond and passes the query appropriately. This process is sometimes referred to as data virtualization.

- **Unfederated Distributed Databases:** In an unfederated distributed database, each node operates individually and there is a central application that manages the access to each database in each node.

- While more complex to manage, heterogeneous distributed databases offer more flexibility in terms of data models, schema choices, and the types of data that can be stored than homogeneous distributed databases.

Features:

- Supports interoperability between diverse systems.
- Complex query optimization and transaction management.
- Useful in mergers or collaborations between organizations.
- **Example:** A logistics company uses MySQL for inventory, MongoDB for vehicle tracking, and PostgreSQL for billing. Integration middleware allows unified querying across these platforms.

(C) Client-Server Distributed Database System:

- In this model, the server stores and manages the database, while clients send queries over the network. It offers centralized control with distributed access, making it ideal for enterprise systems and web applications.
- Clients can be lightweight while the server handles heavy processing.
- **Example:** Web application interacting with a central PostgreSQL server.

Features:

- Simplifies resource management.
- Central servers can be optimized for performance.
- Easily scalable with more clients.

Example: An E-Commerce website where the frontend (client) is hosted separately and interacts with a central PostgreSQL server to manage orders, users, and inventory.

(D) Peer-to-Peer Distributed Database System:

- Here, all nodes are equal, with no fixed client or server roles. Each node can store data and also process queries, leading to decentralized control.
- It supports fault tolerance and high availability.

Example: Blockchain networks like Ethereum, where each node maintains a part of the distributed ledger.

Features:

- No single point of failure.
- Useful in decentralized and distributed apps.
- High availability and data redundancy.

Example: Blockchain-based databases like Ethereum or BitTorrent-based systems, where each peer maintains part of the ledger and participates equally in transactions.

(E) Cloud-Based Distributed Database System:

- These systems are deployed on cloud platforms and span multiple geographic regions for scalability and reliability.
- They abstract infrastructure details and are offered as DBaaS, making them ideal for dynamic workloads. Example: Google Cloud Spanner and Amazon DynamoDB used for global applications.

Features:

- Automatic scaling and replication.
- Pay-as-you-use pricing.
- Global availability and disaster recovery.

Example:

- **Google Cloud Spanner:** Global-scale relational database.
- **Amazon DynamoDB:** Key-value and document database with high performance.
- **Azure Cosmos DB:** Multi-model, globally distributed DBMS.

6.3.5 Key Components and Challenges of a Distributed Database

- Following are key components of Distributed Database System
 - Replication.
 - Fragmentation.
 - Concurrency Control.
 - Semantic Heterogeneity.

(A) Replication:

- As the name suggests, the system stores copies of data at different sites. If an entire database is available on multiple sites, it is a fully redundant database.
- The advantage of data replication is that it increases availability of data on different sites. As the data is available at different sites, queries can be processed parallelly.
- However, data replication has some disadvantages as well. Data needs to be constantly updated and synchronized with other sites, if any site fails to achieve it then it will lead to inconsistencies in the database. Availability of data is highly benefitted from Replication.

- Constant updation complicates concurrency control and it is also overhead for the servers.

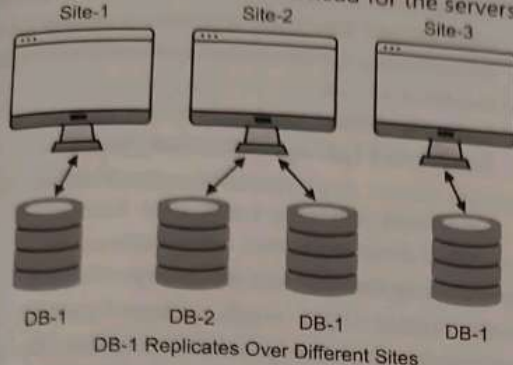


Fig. 6.4: Replication

- Note:** DB-1 replicates on different sites, creating copies of the same data.

(i) Types of Data Replication:

- Data replication is the process by which multiple copies of data are maintained across different nodes, servers, or sites. There are different types of database replication schema to choose from, including:

- Full replication:** In full replication, a complete, functional copy of the entire database is sent to all sites within the distributed database system. Database copy updates are provided on a routine schedule. There are two subtypes of full replication, as well.

- Transactional replication:** In transactional replication, a full and complete database copy is provided to each node, and then data changes are updated to that copy as transaction processing occurs, often in real-time.

- Snapshot replication:** Using snapshot replication, a copy of the database at a specific point in time is captured. This snapshot is then distributed across nodes and the user base as needed but does not consistently monitor for data changes. For this reason, snapshot database replication is only recommended for infrequently changing content.

- Partial replication:** In some cases, certain nodes only require specific portions of the database, so a defined portion of the database is replicated to a select group. In this type of data replication, any number of nodes or sites can receive the replication.

- Merge replication:** As its name indicates, merge database replication is the merging of two databases into one. This is the most complex of the database replication types.

(B) Fragmentation:

- In Fragmentation, the relations are fragmented, which means they are split into smaller parts. Each of the fragments is stored on a different site, where it is required. In this, the data is not replicated, and no copies are created. Consistency of data is highly benefitted from Fragmentation.
- The prerequisite for fragmentation is to make sure that the fragments can later be reconstructed into the original relation without losing any data.
- Consistency is not a problem here as each site has a different piece of information.

- There are two types of fragmentation,

(i) **Horizontal Fragmentation:** Splitting by rows.

(ii) **Vertical fragmentation:** Splitting by columns.

(i) Horizontal Fragmentation (or Sharding)

- The relation schema is fragmented into group of rows, and each group is then assigned to one fragment.

(ii) Vertical Fragmentation:

- The relation schema is fragmented into group of columns, called smaller schemas. These smaller schemas are then assigned to each fragment.
- Each fragment must contain a common candidate key to guarantee a lossless join.

Table 6.1: Fragmentation

(a) Original Table

Customer ID	First Name	Last Name	City
1	Alice	Anderson	Austin
2	Bob	Best	Boston
3	Carrie	Conway	Chicago
4	David	Doe	Denver

(b) Vertical Shards

Customer ID	First Name	Last Name	Customer ID	City
1	Alice	Anderson	1	Austin
2	Bob	Best	2	Boston
3	Carrie	Conway	3	Chicago
4	David	Doe	4	Denver

(c) Horizontal Shards

HS1

Customer ID	First Name	Last Name	City
1	Alice	Anderson	Austin
2	Bob	Best	Boston

HS2

Customer ID	First Name	Last Name	City
3	Carrie	Conway	Chicago
4	David	Doe	Denver

(C) Concurrency Control:

- Concurrency control ensures data remains accurate when multiple transactions run at the same time. Without it, issues like lost updates or dirty reads can occur. Its goal is to make parallel transactions behave as if run one by one. Common methods include locking, timestamps, and optimistic concurrency.

(D) Semantic Heterogeneity:

- Semantic heterogeneity happens when different databases use the same data labels but with different meanings, formats, or units.
- **For example**, one system may store salary in dollars, another in rupees. This can cause confusion during data integration, so resolving it is important for accurate results.

6.3.6 Functions of Distributed Database

- Distributed databases spread data across multiple physical sites/nodes while appearing as a single logical database, with core functions handling data distribution, query processing, replication, and consistency across the network.

(A) Data Distribution (Fragmentation and Allocation):

- Data fragmentation divides tables into horizontal (rows by range/location) or vertical (columns) fragments, then allocates them across nodes based on access patterns or locality.
- **Example:** Employee table fragments by DeptID: Mumbai site holds IT rows (EmpID 101-200), Pune holds HR (201-300).
- **Purpose:** Reduces network traffic and improves local query speed.

(B) Distributed Query Processing:

- Queries are decomposed into subqueries executed in parallel across sites, with results joined via semi-joins or bloom filters to minimize data transfer.
- **Example:** `SELECT * FROM Employee WHERE Salary > 50000` scans Mumbai/IT fragment and Pune/HR fragment concurrently, aggregates locally, sends counts over network. Optimizer chooses site with min cost.

(C) Replication Management:

- Data copies (full/partial) are maintained across sites using synchronous (2PC) or asynchronous (log shipping) methods for redundancy and load balancing.
- **Example:** Primary Mumbai Employee master replicates asynchronously to Pune read-only copy; updates propagate every 5 minutes.
- **Functions:** Failover, read scaling.

(D) Directory Management (Catalog):

- Global/local directories track fragment locations, replicas, and allocation schemas, enabling location transparency (users query without knowing data location).
- **Example:** Global catalog entry:
Employee.IT_frag1 → Mumbai_node1, Mumbai_node2.
- **Two-level scheme:** Local sites know their data; central directory knows mappings.

(E) Concurrency Control and Recovery:

- Distributed 2PL (two-phase locking) and timestamp ordering ensure serializability across sites; commit protocols like 2PC co-ordinate transactions.
- **Example:** Transaction T1 (Mumbai update) locks local fragment, votes YES; coordinator asks Pune replica—if all YES, global COMMIT.
- **Recovery:** Site failures use logs for rollback.

(F) Transparency Services:

- **Fragmentation transparency:** Users unaware of splits (query full table).
- **Replication transparency:** Reads from nearest replica automatically.
- **Location transparency:** No site names in queries.

(G) Key Functions:**Table 6.2**

Function	Purpose	Example Mechanism
Fragmentation	Divide/place data locally	Horizontal range/hash
Query Optimization	Parallel execution, min data transfer	Semi-join reordering
Replication	HA, read scaling	Async log shipping
Directory/Catalog	Location tracking	Global + local dict
Concurrency Control	ACID across sites	Distributed 2PL/2PC
Recovery	Fault tolerance	Distributed logs

6.3.7 Advantages of Distributed Database

- Advantages of distributed database are as follows:
 1. **Better Reliability:** Distributed databases offers better reliability than centralized databases. When database failure occurs in a centralized database, the system comes to a complete stop. But in the

case of distributed databases, the system functions even when a failure occurs, only performance-related issues occur which are negotiable.

2. **Modular Development:** It implies that the system can be expanded by adding new computers and local data to the new site and connecting them to the distributed system without interruption.
3. **Lower Communication Cost:** Locally storing data reduces communication costs for data manipulation in distributed databases. In centralized databases, local storage is not possible.
4. **Better Response Time:** As the data is distributed efficiently in distributed databases, this provides a better response time when user queries are met locally. While in the case of centralized databases, all of the queries have to pass through the central machine which increases response time.

6.3.8 Disadvantages of Distributed Database

Disadvantages of distributed database are as follows:

1. **Costly Software:** Maintaining a distributed database is costly because we need to ensure data transparency, co-ordination across multiple sites which requires costly software.
2. **Large Overhead:** Many operations on multiple sites require complex and numerous calculations, causing a lot of processing overhead.
3. **Improper Data Distribution:** If data is not properly distributed across different sites, then responsiveness to user requests is affected. This in turn increases the response time.

6.4 INTRODUCTION TO NoSQL DATABASE

- NoSQL (Not Only SQL) databases are designed to handle large volumes of unstructured and semi-structured data.
- Unlike traditional relational databases that rely on fixed schemas and tables, NoSQL offers flexible data models and supports horizontal scaling.
- This makes them well-suited for modern applications that require high performance, scalability, and the ability to manage diverse data types efficiently.

6.4.1 Key Features of NoSQL Databases

- **Dynamic schema:** Allow flexible shaping of data to meet new requirements without the need to migrate or change schemas.
- **Horizontal scalability:** They scale horizontally for adding more nodes into the existing ones and acquire enough storage for even bigger datasets and much higher traffic by distributing the load on multiple servers.

Introduction to Modern Databases

- **Document-based:** Data are presented in flexible, semi-structured formats like JSON/BSON (for example, MongoDB).
- **Key-value-based:** They possess a simple but fast access pattern (for example, Redis) by storing data as pairs of keys and values.
- **Column-based:** Data are organized into columns instead of rows (for example, CASSANDRA).
- **Distributed and high availability:** They are designed to be highly available and to automatically handle node failures and data replication across multiple nodes in a database cluster.
- **Flexibility:** Allow developers to store and retrieve data in a flexible and dynamic manner, with support for multiple data types and changing data structures.
- **Performance:** Perfect for big data and real-time analytics and high volume applications.

6.4.2 Challenges of NoSQL Databases

- **Lack of standardization:** NoSQL systems can be vastly different from one another, making it even harder to choose the right one for a specific use case.
- **Lack of ACID compliance:** NoSQL databases may not provide consistency, which is a disadvantage for applications that need strict data integrity.
- **Narrow focus:** Great for storage but lack functionalities as transaction management, in which relational databases are great.
- **Absence of Complex Query Support:** They are not designed to handle complex queries, which means that they are not a good fit for applications that require complex data analysis or reporting.
- **Lack of maturity:** Being relatively new, NoSQL may not have the reliability, security and feature set of traditional relational databases.
- **Management complexity:** For large datasets, maintaining a NoSQL database could be quite more complicated than managing a relational database.
- **Limited GUI Tools:** While some NoSQL databases, like MongoDB offer GUI tools like MongoDB Compass, not all NoSQL databases provide flexible or user-friendly GUI tools.

6.4.3 Types of NoSQL Databases

- There are four common types of NoSQL databases that store data in slightly different ways.

- Each type will provide distinct advantages and disadvantages depending on the dataset.
- In the examples below, we will be using an E-Commerce website to illustrate how each database may store the data.

(A) Key-Value:

- A key-value database consists of individual records organized via key-value pairs. In this model, keys and values can be any type of data, ranging from numbers to complex objects. However, keys must be unique.
- This means this type of database is best when data is attributed to a unique key, like an ID number. Ideally, the data is also simple, and we are looking to prioritize fast queries over fancy features.
- For example, let us say we wanted to store shopping cart information for customers who shop in an e-commerce store. Our key-value database might look like this:

Table 6.3

Key	Value
customer-123	{ "address": "...", name: "...", "preferences": {...} }
customer-456	{ "address": "...", name: "...", "preferences": {...} }

- Amazon DynamoDB and Redis are popular options for developers looking to work with key-value databases.

Features of Key-Value NoSQL:

- **Ultra-low latency:** Sub-millisecond GET/PUT by hash key; in-memory options like Redis hit nanoseconds.
- **Simple API:** Only key operations—no complex queries; values as strings, JSON, binaries, lists, sets.
- **Massive scalability:** Partitioning across 1000s nodes; consistent hashing for even distribution.
- **TTL/eviction:** Automatic expiration for caches, sessions; LRU eviction policies.
- **Pub/sub and streams:** Real-time messaging, leaderboards, rate limiting without secondary indexes.
- **Example:** RedisSET user:123 '{"cart":["item1"], "session":"abc"}' EX 3600 retrieves shopping cart in 0.1ms.

(B) Document:

- A document-based (also called document-oriented) database consists of data stored in hierarchical structures. Some supported document formats include JSON, BSON, XML, and YAML.

- The document-based model is considered an extension of the key-value database and provides querying capabilities not solely based on unique keys. Documents are considered very flexible and can evolve to fit an application's needs.
- They can even model relationships!
- For example, let's say we wanted to store product information for customers who shop in our e-commerce store. A products document might look like this:

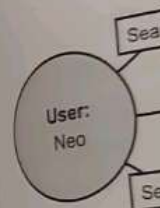
```
[
  {
    "product_title": "Codecademy SQL T-shirt",
    "description": "SQL > NoSQL",
    "link":
      "https://shop.codecademy.com/collections/student-swap/products/sql-tshirt"
    "shipping_details": {
      "weight": 350,
      "width": 10,
      "height": 10,
      "depth": 1
    },
    "sizes": ["S", "M", "L", "XL"],
    "quantity": 1010101010,
    "pricing": {
      "price": 14.99
    }
  }
]
```

- MongoDB is a popular option for developers looking to work with a document database.

Features of Document NoSQL:

- **Flexible schema:** Self-contained JSON/BSON documents with nested arrays/objects; schema evolves per document.
- **Rich querying:** Dot notation (user.address.city), regex, geospatial, full-text search, aggregation pipelines.
- **Atomic updates:** Embed or reference related data; multi-document ACID transactions in modern stores.
- **Horizontal sharding:** Automatic partitioning by document key; secondary indexes for flexible queries.
- **Schema-less evolution:** Add fields without migrations—ideal for evolving APIs, user profiles, catalogs.

- **Graph:** A graph database structure. In a graph database, individual nodes establish relationships (links or lines). The advantage of a graph database is that it can manage and recommend items our store.



- In the graph database, nodes are connected by edges. For example, a user node might be connected to a search node and a recommendations node.

Features of Graph NoSQL:

- **Index-free adjacency:** Nodes are directly connected to their neighbors, eliminating the need for an index.
- **Proximity queries:** Easy to find nodes within a certain distance from a given node.

- **Example:** MongoDB storing products (name:"Laptop", specs:{ram:16, gpu:"RTX"}, tags:["gaming"]) queried by any nested field.

(C) Graph:

- A graph database stores data using a graph structure. In a graph structure, data is stored in individual nodes (also called vertices) and establishes relationships via edges (also called links or lines).
- The advantage of the relationships built using a graph database as opposed to a relational database is that they are much simpler to set-up, manage, and query.
- For example, let us say we wanted to build a recommendation engine for our E-Commerce store.
- We could establish relationships between similar items our customers searched for to create recommendations.

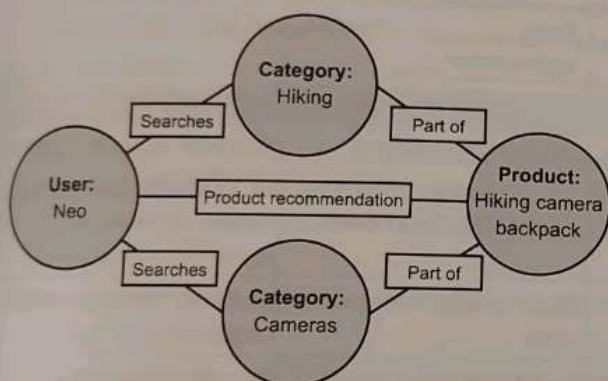


Fig. 6.5: Graph NoSQL Data Base

- In the graph above, we can see that there are four nodes: "Neo", "Hiking", "Cameras", and "Hiking Camera Backpack". Because the user, "Neo", searched for "Hiking" and "Cameras", there are edges connecting all 3 nodes. More edges are created after the search, linking a new node, "Hiking Camera Backpack".
- Neo4j is a popular option for developers looking to work with a graph database.

Features of Graph NoSQL:

- **Index-free adjacency:** Edges stored directly on nodes for O(1) traversals—no JOIN explosion like relational graphs.
- **Property graphs:** Nodes/edges carry key-value properties; supports directed/undirected, weighted relationships.

Introduction to Modern Databases

- **Traversal languages:** Cypher (Neo4j) or Gremlin (path).
- **Native indexing:** Composite indexes on properties; bidirectional edge traversal in milliseconds.
- **Real-time analytics:** Depth-first/breadth-first traversals for recommendations, fraud rings, social networks.
- **Example:** Neo4j query `MATCH (user)-[:PURCHASED]->(product)-[:PURCHASED]-(other) RETURN other` finds similar buyers instantly.

(D) Column Oriented:

- A column-oriented NoSQL database stores data similar to a relational database. However, instead of storing data as rows, it is stored as columns.
- Column-oriented databases aim to provide faster read speeds by being able to quickly aggregate data for a specific column.
- For example, take a look at the following E-Commerce database of products:

Table 6.5: Row Oriented Database

ID	Product Name	Total Sales	Sizes
1	Red Shirt	532	3
2	Jeans	774	10
3	Blue Shirt	861	3

Table 6.6: Column Oriented Database

ID	Product Name
1	Red Shirt
2	Jeans
3	Blue Shirt

ID	Total Sales
1	532
2	774
3	861

ID	Sizes
1	3
2	10
3	3

- If we wanted to analyze the total sales for all the products, all we would need to do is aggregate data from the sales column.
- This is in contrast to a relational model that would have to pull data from each row. We would also be pulling adjacent data (like size information in the above example) that is not relevant to our query.

- Amazon's Redshift is a popular option for developers looking to work with a column-oriented database.

Features of Column-Oriented NoSQL

- Columnar storage:** Data organized by column families (groups of related columns) rather than rows, enabling efficient compression (10 : 1 ratios) and fast analytics scans.
- Dynamic schema:** Sparse columns with varying row lengths; add / remove columns without full table rewrites.

- Aggregation speed:** Optimized for SUM, AVG, COUNT on large datasets—reads only required columns, skipping irrelevant data.
- Horizontal scalability:** Sharding by row key across thousands of nodes; supports massively parallel processing (MPP).
- Time-series excellence:** Handles sequential data (logs, metrics) with bloom filters and SSTables for quick range queries.
- Example:** Cassandra querying "avg temperature by sensor last 24h" scans compressed timestamp/metric columns only.

6.4.4 Compare Different types of NoSQL

Table 6.7

Aspect	Column-Oriented	Graph	Document	Key-Value
Data Model	Rows → dynamic columns/families	Nodes → edges/properties	JSON/BSON documents	Simple key → opaque value
Schema	Semi-structured (column families)	Schema-free (properties vary)	Flexible (nested fields)	No schema (value is blob)
Query Style	Column scans, aggregations	Traversal (Cypher/Gremlin)	Field queries, aggregation	GET/PUT by key only
Best For	Analytics, time-series	Relationships, networks	Semi-structured apps	Caching, sessions
Scalability	Horizontal (sharding by row key)	Vertical + relationship scaling	Horizontal sharding	Extreme horizontal (partitions)
Examples	Cassandra, HBase, BigTable	Neo4j, Amazon Neptune	MongoDB, Couchbase	Redis, DynamoDB
Strength	Column compression, fast scans	Deep traversals (friends-of)	Rich queries on nested data	Sub-ms latency, simple ops
Weakness	Poor for point updates	Complex for non-graph data	Joins expensive	No complex queries

6.4.5 Differences between SQL and NoSQL

Table 6.8

Aspect	SQL (Relational)	NoSQL (Non-relational)
Data Structure	Tables with rows and columns	Document-based, key-value, column-family, or graph-based
Schema	Fixed schema (predefined structure)	Flexible schema (dynamic and adaptable)
Scalability	Vertically scalable (upgrading hardware)	Horizontally scalable (adding more servers)
Data Integrity	ACID-compliant (strong consistency)	BASE-compliant (more available, less consistent)

Query Language	SQL (Structured Query Language)	Varies (For example, MongoDB uses its own query language)
Performance	Efficient for complex queries and transactions	Better for large-scale data and fast read/write operations
Use Case	Best for transactional systems (banking, ERP etc.)	Ideal for big data, real-time web apps, and data lakes
Examples	MySQL, PostgreSQL, Oracle, MS SQL Server	MongoDB, Cassandra, CouchDB, Neo4j

6.4.6 Common Use Cases for NoSQL Databases

- **Web applications:** NoSQL databases are often used to power web applications, which require scalability, performance, and flexibility.
- **Big Data:** NoSQL databases are commonly used in big data applications, where traditional relational databases can struggle to handle the massive volumes of data involved.
- **Internet of Things (IoT):** NoSQL databases are used to store and process data from IoT devices, which can generate massive amounts of data in real time.
- **Real-Time Analytics:** NoSQL databases can be used for real-time analytics, enabling businesses to make faster, data-driven decisions.
- **Content Management:** NoSQL databases are often used for content management applications, which require the ability to handle unstructured or semi-structured data such as documents, images, and videos.

6.5 OBJECT-ORIENTED DATABASES

- The **ODDBMS** which is an abbreviation for **object-oriented database management system** is the data model in which data is stored in form of objects, which are instances of classes.
- These classes and objects together make an object-oriented data model.

6.5.1 Components of Object-Oriented Data Model

- The OODBMS is based on three major components, namely: Object structure, Object classes, and Object identity. These are explained below:

(A) Object Structure:

- The structure of an object refers to the properties that an object is made up of. These properties of an object are referred to as an attribute.
- Thus, an object is a real-world entity with certain attributes that makes up the object structure.
- Also, an object encapsulates the data code into a single unit which in turn provides data abstraction by hiding the implementation details from the user.

The object structure is further composed of three types of components: Messages, Methods, and Variables. These are explained below.

1. Messages:

- A message provides an interface or acts as a communication medium between an object and the outside world.
- A message can be of two types:
 - (i) **Read-only message:** If the invoked method does not change the value of a variable, then the invoking message is said to be a read-only message.
 - (ii) **Update message:** If the invoked method changes the value of a variable, then the invoking message is said to be an update message.

2. Methods:

- When a message is passed then the body of code that is executed is known as a method. Whenever a method is executed, it returns a value as output.
- A method can be of two types:
 - (i) **Read-only method:** When the value of a variable is not affected by a method, then it is known as the read-only method.
 - (ii) **Update-method:** When the value of a variable change by a method, then it is known as an update method.
- **Variables:**
It stores the data of an object. The data stored in the variables makes the object distinguishable from one another.

(B) Object Classes:

- An object which is a real-world entity is an instance of a class. Hence first we need to define a class and then the objects are made which differ in the values they store but share the same class definition.
- The objects in turn correspond to various messages and variables stored in them.

Example:

```
class CLERK
{ //variables
  char name;
  string address;
  int id;
  int salary;
  //Messages
  charget_name();
  stringget_address();
  intannual_salary();
};
```


- In the above example, we can see, CLERK is a class that holds the object variables and messages. An OODBMS also supports inheritance in an extensive manner as in a database there may be many classes with similar methods, variables and messages. Thus, the concept of the class hierarchy is maintained to depict the similarities among various classes.
- The concept of encapsulation that is the data or information hiding is also supported by an object-oriented data model. And this data model also provides the facility of abstract data types apart from the built-in data types like char, int, float. ADT's are the user-defined data types that hold the values within them and can also have methods attached to them. Thus, OODBMS provides numerous facilities to its users, both built-in and user-defined. It incorporates the properties of an object-oriented data model with a database management system, and supports the concept of programming paradigms like classes and objects along with the support for other concepts like encapsulation, inheritance, and the user-defined ADT's (abstract data types).
- ODBMS stands for Object-Oriented Database Management System, which is a type of database management system that is designed to store and manage object-oriented data. Object-oriented data is data that is represented using objects, which encapsulate data and behavior into a single entity.
- An ODBMS stores and manages data as objects, and provides mechanisms for querying, manipulating, and retrieving the data. In an ODBMS, the data is typically stored in the form of classes and objects, which can be related to each other using inheritance and association relationships.
- In an ODBMS, the data is managed using an object-oriented programming language or a specialized query language designed for object-oriented databases. Some of the popular object-oriented database languages include Smalltalk, Java, and C++. Some ODBMS also support standard SQL for querying the data.
- ODBMS have several advantages over traditional relational databases. One of the main advantages is that they provide a natural way to represent complex data structures and relationships. Since the data is represented using objects, it can be easier to model real-world entities in the database.

Additionally, ODBMS can provide better performance and scalability for applications that require a large number of small, complex transactions.

- However, there are also some disadvantages to using an ODBMS. One of the main disadvantages is that they can be more complex and harder to use than traditional relational databases. Additionally, ODBMS may not be as widely used and supported as traditional relational databases, which can make it harder to find expertise and support. Finally, some applications may not require the advanced features and performance provided by an ODBMS, and may be better suited for a simpler database solution.

6.5.2 Features of ODBMS

- **Object-oriented data model:** ODBMS uses an object-oriented data model to store and manage data. This allows developers to work with data in a more natural way, as objects are similar to the objects in the programming language they are using.
- **Complex data types:** ODBMS supports complex data types such as arrays, lists, sets, and graphs, allowing developers to store and manage complex data structures in the database.
- **Automatic schema management:** ODBMS automatically manages the schema of the database, as the schema is defined by the classes and objects in the application code. This eliminates the need for a separate schema definition language and simplifies the development process.
- **High performance:** ODBMS can provide high performance, especially for applications that require complex data access patterns, as objects can be retrieved with a single query.
- **Data integrity:** ODBMS provides strong data integrity, as the relationships between objects are maintained by the database. This ensures that data remains consistent and correct, even in complex applications.
- **Concurrency control:** ODBMS provides concurrency control mechanisms that ensure that multiple users can access and modify the same data without conflicts.
- **Scalability:** ODBMS can scale horizontally by adding more servers to the database cluster, allowing it to handle large volumes of data.
- **Support for transactions:** ODBMS supports transactions, which ensure that multiple operations on the database are atomic and consistent.

6.5.3 Advantages

- Advantages

- **Support for complex data types:** ODBMS supports complex data types such as arrays, lists, sets, and graphs, allowing developers to store and manage complex data structures in the database.
- **Automatic schema management:** ODBMS automatically manages the schema of the database, as the schema is defined by the classes and objects in the application code. This eliminates the need for a separate schema definition language and simplifies the development process.
- **High performance:** ODBMS can provide high performance, especially for applications that require complex data access patterns, as objects can be retrieved with a single query.
- **Data integrity:** ODBMS provides strong data integrity, as the relationships between objects are maintained by the database. This ensures that data remains consistent and correct, even in complex applications.
- **Concurrency control:** ODBMS provides concurrency control mechanisms that ensure that multiple users can access and modify the same data without conflicts.
- **Scalability:** ODBMS can scale horizontally by adding more servers to the database cluster, allowing it to handle large volumes of data.
- **Support for transactions:** ODBMS supports transactions, which ensure that multiple operations on the database are atomic and consistent.

6.5.4 Disadvantages

- Disadvantages

- **Complexity:** ODBMS can be more complex and harder to use than traditional relational databases. Additionally, ODBMS may not be as widely used and supported as traditional relational databases, which can make it harder to find expertise and support.
- **Performance:** ODBMS may not provide high performance for all applications. For example, ODBMS may not be as fast as traditional relational databases for simple queries.
- **Data integrity:** ODBMS may not provide strong data integrity for all applications. For example, ODBMS may not be as good at maintaining relationships between objects as traditional relational databases.
- **Concurrency control:** ODBMS may not provide strong concurrency control for all applications. For example, ODBMS may not be as good at preventing conflicts between users as traditional relational databases.
- **Scalability:** ODBMS may not be as scalable as traditional relational databases. For example, ODBMS may not be as good at handling large volumes of data as traditional relational databases.
- **Support for transactions:** ODBMS may not provide strong support for transactions. For example, ODBMS may not be as good at ensuring that multiple operations on the database are atomic and consistent as traditional relational databases.

6.5.3 Advantages of OODBs

Advantages of OODBs are as follows:

- **Supports Complex Data Structures:** ODBMS is designed to handle complex data structures, such as inheritance, polymorphism, and encapsulation. This makes it easier to work with complex data models in an object-oriented programming environment.
- **Improved Performance:** ODBMS provides improved performance compared to traditional relational databases for complex data models. ODBMS can reduce the amount of mapping and translation required between the programming language and the database, which can improve performance.
- **Reduced Development Time:** ODBMS can reduce development time since it eliminates the need to map objects to tables and allows developers to work directly with objects in the database.
- **Supports Rich Data Types:** ODBMS supports rich data types, such as audio, video, images, and spatial data, which can be challenging to store and retrieve in traditional relational databases.
- **Scalability:** ODBMS can scale horizontally and vertically, which means it can handle larger volumes of data and can support more users.

6.5.4 Disadvantages of OODBs

Disadvantages of OODBs are as follows:

- **Limited Adoption:** ODBMS is not as widely adopted as traditional relational databases, which means it may be more challenging to find developers with experience working with ODBMS.
- **Lack of Standardization:** ODBMS lacks standardization, which means that different vendors may implement different features and functionality.
- **Cost:** ODBMS can be more expensive than traditional relational databases since it requires specialized software and hardware.
- **Integration with Other Systems:** ODBMS can be challenging to integrate with other systems, such as business intelligence tools and reporting software.
- **Scalability Challenges:** ODBMS may face scalability challenges due to the complexity of the data models it supports, which can make it challenging to partition data across multiple nodes.

6.5.5 Real-World Applications of OODBs

- In some industries and systems, object-oriented databases are particularly beneficial because they can efficiently work with complex data structures and relationships.
 - **Computer-aided design and computer-aided manufacturing:** In computer-aided design and computer-aided manufacturing, OODBs excel in managing intricate design models and manufacturing processes. They can store detailed geometric and structural data as objects, making managing and modifying complex designs easier.
 - **Telecommunications:** The telecommunications industry benefits from OODBs' ability to handle vast amounts of interconnected data. They are ideal for managing network configurations, call records, and customer information, where relationships between data points are crucial.
 - **Scientific research:** In scientific research, particularly in fields like bioinformatics, physics, and environmental science, OODBs can manage large datasets with complex relationships. They allow researchers to model and analyze data more effectively, facilitating discoveries and innovations.
 - **Multimedia applications:** OODBs are well-suited for multimedia applications that require storing and retrieving images, videos, and audio files metadata. They can manage these data types as objects, making organizing and accessing multimedia content easier.

EXERCISE

➤ 3 Marks Questions

1. Define modern databases.
2. What are recent trends in DBMS?
3. Define a cloud database.
4. List any two features of cloud databases.
5. What is meant by pay-as-you-go pricing in cloud databases?
6. Define a distributed database.
7. State the need for distributed databases.
8. What is data replication?
9. Define fragmentation in distributed databases.
10. What is horizontal fragmentation?
11. Define NoSQL database.
12. List any two features of NoSQL databases.
13. What is a key-value database?
14. What is a document-oriented database?
15. Define object-oriented database (OODBMS).